

새싹 강동캠퍼스 베스핀글로벌

AWS RDS Multi-AZ

마이그레이션 & Failover 수행평가

MISSION 2 | 기존 3-Tier 환경에서 DB 계층만 고도화

EC2 MySQL

→ RDS Multi-AZ →

Failover Test

강사 젠아이랩스 박수현

기존 VPC/Subnet/NAT/Bastion/WEB/WAS 는 그대로 사용합니다.

시나리오

고객사 시나리오

A 사는 이전 미션에서 AWS VPC 기반 3-Tier 서비스를 구축하여 WEB → WAS → EC2 MySQL 구조로 상품 조회 서비스를 운영하고 있습니다. 서비스는 정상 동작하지만, 데이터베이스를 EC2 에서 직접 운영하면서 장애 대응, 백업, 유지관리 부담이 커졌습니다. 고객사는 기존 네트워크와 애플리케이션 구조를 최대한 유지한 채 DB 계층만 Amazon RDS for MySQL 로 전환하고, Multi-AZ 를 적용해 장애 발생 시 다른 AZ 로 자동 전환되는 구조를 원합니다. 기존 products 데이터는 그대로 보존되어야 하며, 전환 완료 후 강제 Failover 를 수행하여 애플리케이션의 자동 복구까지 확인해야 합니다.

이번 미션의 전제

- Mission 1 의 VPC 10.0.0.0/16, 6 개 Subnet, NAT Gateway, Route Table, Bastion, WEB/WAS 서버를 그대로 사용합니다.
- 기존 DB-A EC2 에는 MySQL 과 testdb.products 데이터가 존재하며 WEB-A/WEB-C 에서 상품 목록이 정상 출력되는 상태에서 시작합니다.
- 이번 미션에서는 새 VPC 나 새 WEB/WAS 서버를 만들지 않습니다.
- 기존 DB-A EC2 는 마이그레이션과 Failover 검증이 끝날 때까지 삭제하지 않습니다.

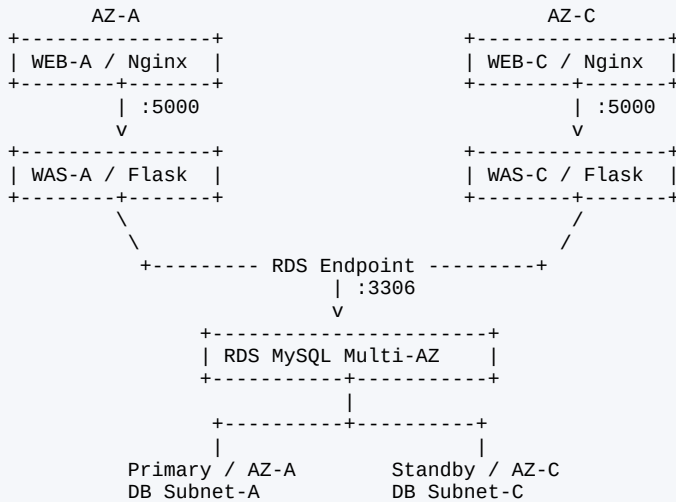
Mission Goal

```
EC2 MySQL  -> mysqldump  -> RDS MySQL Multi-AZ
                        |
                        Same RDS Endpoint
                        |
                    Forced Failover Test
```

1. 목표 아키텍처

이번 미션은 기존 네트워크를 재구축하지 않고 DB 계층만 교체합니다. WAS-A 와 WAS-C 는 동일한 RDS Endpoint 를 사용합니다.

Existing VPC 10.0.0.0/16



Migration: WAS-A -> Old DB-A EC2 -> testdb.sql -> RDS

중요

이번 미션에서는 “RDS Multi-AZ DB Instance”를 사용합니다. Multi-AZ DB Cluster 와 혼동하지 않습니다. Standby 는 평상시 읽기 서버로 사용하는 대상이 아니며, 애플리케이션은 하나의 RDS Endpoint 만 사용합니다.

2. Step 1 - DB Subnet Group 과 Security Group

2.1 기존 DB Subnet 을 DB Subnet Group 으로 묶기

RDS 가 두 Availability Zone 을 사용할 수 있도록 Mission 1 에서 이미 만든 DB Subnet 2 개를 재사용합니다.

항목	설정
DB Subnet Group 이름	sesac-db-subnet-group
VPC	기존 VPC 10.0.0.0/16
AZ-A Subnet	DB-A : 10.0.12.0/24
AZ-C Subnet	DB-C : 10.0.22.0/24

체크 포인트

RDS DB Subnet Group 은 최소 2 개의 Availability Zone 에 걸친 Subnet 을 포함해야 합니다. 이번에는 기존 DB-A, DB-C Subnet 을 그대로 사용합니다.

2.2 SG-RDS 생성

Protocol	Port	Source	결과
TCP	3306	SG-WAS	허용
TCP	3306	SG-WEB	Rule 없음 / 차단
TCP	3306	SG-BASTION	Rule 없음 / 차단
TCP	3306	0.0.0.0/0	절대 추가하지 않음

RDS 는 SSH 접속 대상이 아니므로 22 번 포트를 열지 않습니다. 데이터 마이그레이션도 WAS-A 를 통해 수행합니다.

3. Step 2 - RDS MySQL Multi-AZ 생성

설정 항목	권장 설정
Engine	MySQL 8.0 계열 (콘솔에서 제공되는 버전)
Deployment	Multi-AZ DB instance
DB identifier	sesac-rds-mysql
Master user	admin
DB class	db.t3.micro 또는 수업에서 지정한 소형 Burstable 클래스
Storage	20 GiB, General Purpose (예: gp3)
VPC	Mission 1 의 기존 VPC
DB Subnet Group	sesac-db-subnet-group
Public access	No
VPC Security Group	SG-RDS

비용 주의

Multi-AZ DB Instance 는 Primary 와 Standby 가 함께 구성되므로 Single-AZ 보다 비용이 증가합니다. 평가 완료 후 강사가 지정한 절차에 따라 불필요한 RDS 와 Snapshot 을 정리합니다.

RDS 생성 후 기록할 값

- ☐ RDS Endpoint: _____
- ☐ Port: 3306
- ☐ 현재 Primary Availability Zone: _____
- ☐ Secondary Availability Zone: _____

4. Step 3 - 기존 EC2 MySQL 데이터 마이그레이션

Migration Host 는 WAS-A 를 사용합니다. 기존 Security Group 정책을 바꾸지 않고 Old DB-A 와 RDS 양쪽에 접근할 수 있기 때문입니다.

4.1 WAS-A 에 MySQL Client 설치

```
sudo apt update
sudo apt install -y mysql-client
```

4.2 기존 DB-A 에서 testdb Dump

```
mysqldump -h <DB-A-PRIVATE-IP> -u webuser -p testdb > testdb.sql
ls -lh testdb.sql
```

4.3 RDS 에 Database 생성 및 Import

```
mysql -h <RDS-ENDPOINT> -u admin -p -e "CREATE DATABASE testdb CHARACTER SET utf8mb4;"
mysql -h <RDS-ENDPOINT> -u admin -p testdb < testdb.sql
```

4.4 애플리케이션 계정 생성

```
mysql -h <RDS-ENDPOINT> -u admin -p

CREATE USER 'webuser'@'10.0.0.0/24' IDENTIFIED BY 'password';
GRANT SELECT ON testdb.* TO 'webuser'@'10.0.0.0/24';
FLUSH PRIVILEGES;
```

왜 SELECT 만 허용하나요?

현재 Mission 1 의 Flask 예제는 products 목록을 SELECT 하는 기능만 사용합니다. 따라서 이번 미션에서는 최소 권한 원칙에 따라 SELECT 만 부여합니다. 쓰기 기능이 추가되면 필요한 권한만 추가합니다.

5. Step 4 - Flask 를 RDS Endpoint 로 전환

WAS-A 와 WAS-C 의 app.py 에서 DB Host 만 변경합니다. WEB/Nginx 설정은 변경하지 않습니다.

변경 전

```
host="10.0.12.10"
```

변경 후

```
host="<RDS-ENDPOINT>"
```

두 WAS 서버 모두 동일한 RDS Endpoint 를 사용합니다. Multi-AZ Failover 가 발생해도 애플리케이션의 Host 문자열은 바뀌지 않습니다.

5.1 데이터 확인

```
mysql -h <RDS-ENDPOINT> -u webuser -p testdb -e "SELECT * FROM products;"
```

5.2 서비스 확인

- WEB-A Public IP 접속 → 기존 상품 목록이 출력되는지 확인
- WEB-C Public IP 접속 → 동일한 상품 목록이 출력되는지 확인
- WAS-A → RDS :3306 연결 성공
- WAS-C → RDS :3306 연결 성공
- WEB → RDS :3306 연결 실패
- Bastion → RDS :3306 연결 실패

```
nc -zv <RDS-ENDPOINT> 3306
```

위 명령을 WAS 에서는 성공, WEB/Bastion 에서는 실패하도록 확인합니다.

6. Step 5 - 강제 Failover 테스트

평가 목표

RDS의 Primary가 다른 AZ로 전환되는 동안 서비스가 일시적으로 실패할 수 있지만, Failover 완료 후 Flask 코드의 Endpoint를 변경하지 않고 다시 정상 동작해야 합니다.

6.1 Failover 전 기록

- ☐ RDS Endpoint를 기록했다.
- ☐ 현재 Primary AZ를 기록했다.
- ☐ WEB-A / WEB-C에서 상품 목록이 정상 출력된다.

6.2 WEB-A에서 서비스 상태 반복 확인

```
while true; do date '+%H:%M:%S'; curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://localhost; sleep 2; done
```

6.3 RDS Console에서 강제 Failover

1. RDS → Databases → sesac-rds-mysql 선택
2. Actions → Reboot 선택
3. Reboot with failover 옵션을 선택하여 실행
4. 반복 curl 결과에서 일시적인 실패/복구를 관찰

6.4 Failover 완료 후 확인

- ☐ 서비스가 다시 HTTP 200으로 복구되었다.
- ☐ RDS Endpoint 문자열은 Failover 전과 동일하다.
- ☐ Primary AZ가 다른 AZ로 변경되었음을 확인했다.
- ☐ Flask app.py의 host 값을 수정하지 않았다.

관찰 포인트

Multi-AZ DB Instance의 Failover는 일반적으로 약 60~120초가 걸릴 수 있으며 DB 상태에 따라 더 길어질 수 있습니다. RDS Console의 AZ 표시는 실제 Failover 완료보다 늦게 갱신될 수 있습니다.

7. 제출 결과 및 평가 Sheet

필수 증빙 화면

- ☐ DB Subnet Group 에 DB-A / DB-C Subnet 이 포함된 화면
- ☐ SG-RDS : TCP 3306 Source = SG-WAS 화면
- ☐ RDS MySQL Multi-AZ 및 Public access = No 확인 화면
- ☐ 기존 EC2 MySQL 의 products 데이터 Dump/Import 결과
- ☐ RDS 에서 SELECT * FROM products 결과
- ☐ WEB-A / WEB-C 상품 목록 정상 출력 화면
- ☐ Failover 실행 전/후 Primary AZ 확인 화면
- ☐ Failover 후 동일 Endpoint 로 서비스가 복구된 화면

평가 기준

평가 항목	배점
기존 Mission 1 환경을 그대로 재사용하고 사전 상태 확인	10
DB Subnet Group + SG-RDS 최소 권한 구성	20
RDS MySQL Multi-AZ 생성	20
mysqldump / Import 를 통한 products 데이터 이전	20
WAS-A / WAS-C 의 RDS Endpoint 전환 및 서비스 검증	15
강제 Failover 실행 및 자동 복구 검증	15
총점	100

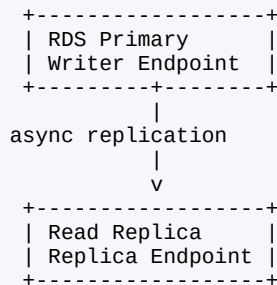
완료 조건

“RDS 가 생성되었다”가 아니라 기존 products 데이터가 유지되고, WEB-A/WEB-C 모두 정상 조회되며, 강제 Failover 이 후에도 코드의 Endpoint 변경 없이 서비스가 복구되어야 Mission Complete 입니다.

부록 A. Read Replica 는 언제 사용하는가?

Read Replica 는 Multi-AZ Standby 와 목적이 다릅니다. 이번 본 미션의 필수 구성은 Multi-AZ 이며, 아래 내용은 읽기 부하가 커지는 경우를 위한 심화 참고입니다.

구분	RDS Multi-AZ DB Instance	Read Replica
주 목적	고가용성 / 자동 Failover	읽기 성능 확장
복제 특성	Standby 로 동기식 복제	비동기 복제
평상시 읽기	Standby 직접 사용 불가	Replica 에서 SELECT 가능
Endpoint	애플리케이션은 Primary Endpoint 1 개	Replica 마다 별도 Endpoint
장애 시	RDS 가 Standby 로 자동 Failover	자동 Primary 대체 용도가 아님
주의점	Failover 중 일시 단절	Replication Lag 가능



Application rule:

SELECT -> Read Replica Endpoint
 INSERT/UPDATE/DELETE -> Primary Endpoint

핵심

Read Replica 는 “2 중화니까 자동 장애조치”라고 이해하면 안 됩니다. 일반적인 RDS DB Instance Read Replica 는 읽기 부하 분산용이며, Source 와 별도의 Endpoint 를 갖습니다. Source 장애 시 애플리케이션이 Replica 를 자동으로 Writer 처럼 사용하는 구조가 아닙니다.

부록 B. Read Replica 애플리케이션 설정 예제

현재 실습 애플리케이션은 products 를 SELECT 만 하므로, Read Replica 를 실습한다면 / 경로의 조회 연결만 Replica Endpoint 로 보내는 정도로 충분합니다.

1) Endpoint 를 두 개 준비

```
DB_WRITER = "<PRIMARY-RDS-ENDPOINT>"
DB_READER = "<READ-REPLICA-ENDPOINT>"
```

2) 현재 상품 조회는 Reader 사용

```
conn = pymysql.connect(
    host=DB_READER,
    user="webuser",
    password="password",
    database="testdb"
)

cursor.execute("SELECT * FROM products")
```

3) 쓰기 기능이 생기면 Writer 사용

```
# INSERT / UPDATE / DELETE 는 Primary Endpoint 사용
conn = pymysql.connect(
    host=DB_WRITER,
    user="webuser",
    password="password",
    database="testdb"
)
```

간단한 Read Replica 확인 실습

1. RDS Primary 에서 Actions → Create read replica 로 Replica 를 생성합니다.
2. 생성된 Read Replica 의 별도 Endpoint 를 기록합니다.
3. Flask 의 SELECT 연결을 Replica Endpoint 로 변경하여 상품 목록이 동일하게 조회되는지 확인합니다.
4. Primary 에서 새 데이터를 추가한 직후 Replica 에서 조회하여 복제 지연(Replication Lag)이 발생할 수 있음을 관찰합니다.

개발 과정이 아닌 경우

본 과정에서는 Writer/Reader 라우팅 로직을 복잡하게 구현하지 않습니다. “SELECT 는 Replica, 쓰기는 Primary”라는 원칙과 Endpoint 가 분리된다는 점만 이해하면 충분합니다.

부록 C. 참고 및 정리

이번 미션에서 반드시 기억할 것

- 기존 3-Tier VPC 는 그대로 두고 DB 계층만 EC2 MySQL 에서 RDS 로 전환한다.
- RDS DB Subnet Group 은 서로 다른 최소 2 개 AZ 의 Subnet 을 사용한다.
- SG-RDS 의 3306 Source 는 SG-WAS 만 허용한다.
- WAS-A 와 WAS-C 는 동일한 RDS Endpoint 를 사용한다.
- Multi-AZ Standby 는 읽기 서버가 아니다.
- Failover 테스트 후에도 Endpoint 문자열은 그대로이며 서비스가 자동 복구되어야 한다.
- Read Replica 는 HA 보다 Read Scaling 이 목적이며 Replica Endpoint 를 애플리케이션이 명시적으로 사용해야 한다.

AWS 공식 문서 참고

RDS DB instance 생성 / DB Subnet Group

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateDBInstance.html

Multi-AZ Failover

<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.Failover.html>

DB Instance Reboot with Failover

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_RebootInstance.html

MySQL Multi-AZ 와 Read Replica

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_MySQL.Replication.ReadReplicas.MultiAZ.html

MISSION 2 COMPLETE

EC2 MySQL 의 기존 데이터를 RDS Multi-AZ 로 안전하게 이전하고, 두 WAS 가 동일 Endpoint 를 사용하며, 강제 Failover 후에도 서비스가 다시 정상화되면 미션 완료입니다.